

Лабораторная работа №8

**Дискретно-событийное моделирование распространения
инфекции (SIR-модель)**

Еремина Оксана Андреевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Дискретно-событийное моделирование	7
3.2	SIR-модель эпидемии	7
3.3	Детерминированная SIR-модель	8
3.4	Стохастическая SIR-модель	8
3.5	Базовое репродуктивное число	8
3.6	Используемые программные средства	9
4	Выполнение лабораторной работы	10
5	Обзор sir_des	12
5.1	Подключение библиотек и модулей	12
5.2	Параметры модели по умолчанию	13
5.3	Базовый запуск модели	14
5.4	Анализ чувствительности к параметрам (Задание 8.5.1)	15
5.5	Сравнение стохастической и детерминированной версий (Задание 8.5.2)	20
5.6	Оценка производительности (Задание 8.5.3)	21
5.7	Сохранение результатов в CSV (Задание 8.5.4)	22
5.8	Итоговая статистика	23
5.9	Выводы	24
6	Вывод	26
	Список литературы	27

Список иллюстраций

4.1	Запуск кода и его вывод	10
4.2	Запуск кода и его вывод	11
4.3	Компиляция sir_des.jl	11

Список таблиц

1 Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

2 Задание

1. Создать рабочий каталог для кода
2. Установить необходимые пакеты Julia
3. Выполнить предложенный код SIR-модели
4. Преобразовать код в литературный стиль (Literate.jl)
5. Сгенерировать из литературного кода:
 - чистый код
 - Jupyter notebook
 - документацию в формате Quarto
6. Выполнить код из Jupyter notebook
7. Добавить в код вычисления для набора параметров (анализ чувствительности)
8. Сгенерировать обновленные файлы с параметрами
9. Интегрировать документацию Quarto в отчет

3 Теоретическое введение

3.1 Дискретно-событийное моделирование

Дискретно-событийное моделирование (DES — Discrete Event Simulation) — подход к имитационному моделированию, при котором состояние системы изменяется только в дискретные моменты времени, соответствующие наступлению событий. В отличие от непрерывного моделирования, где время изменяется с фиксированным шагом, DES использует продвижение от события к событию, что позволяет эффективно моделировать системы с редкими изменениями состояния [1].

3.2 SIR-модель эпидемии

SIR-модель является классической математической моделью эпидемиологии, описывающей распространение инфекционного заболевания в популяции [3]. Популяция разделяется на три группы:

- **S (Susceptible)** — восприимчивые к заболеванию индивиды
- **I (Infected)** — инфицированные и заразные индивиды
- **R (Recovered/Removed)** — переболевшие (или удаленные из популяции) индивиды

3.3 Детерминированная SIR-модель

Система обыкновенных дифференциальных уравнений для детерминированной SIR-модели имеет вид:

$$\frac{dS}{dt} = -\frac{\beta c}{N}SI$$

$$\frac{dI}{dt} = \frac{\beta c}{N}SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

где $N = S + I + R$ — общая численность популяции.

3.4 Стохастическая SIR-модель

В стохастической версии модели, реализованной в данной работе [4]:

- Время между контактами распределено экспоненциально с параметром c (частота контактов)
- Вероятность заражения при контакте с инфицированным равна β
- Время выздоровления распределено экспоненциально с параметром γ

3.5 Базовое репродуктивное число

Ключевой параметр эпидемии — базовое репродуктивное число R_0 :

$$R_0 = \frac{\beta c}{\gamma}$$

Интерпретация:

- Если $R_0 > 1$ — эпидемия распространяется

- Если $R_0 < 1$ — эпидемия затухает

3.6 Используемые программные средства

Реализация выполнена на языке программирования Julia [2] с использованием пакетов `ConcurrentSim` (дискретно-событийное моделирование), `ResumableFunctions` (возобновляемые функции) и `Distributions` (генерация случайных величин).

4 Выполнение лабораторной работы

Создала необходимые файлы, куда скопировала весь код, предоставленный в лабораторной работе.

Запустила их всех (рис. 4.1, рис. 4.2).

```
zrdagdelen@zrdagdelen-MacBookPro:~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project$ julia --project=. scripts/sir_des.jl
=====
ДИСКРЕТНО-СОБЫТИЙНАЯ SIR МОДЕЛЬ
=====

1. Базовый запуск модели...
   График сохранён: plots/sir_des.png

2. Анализ чувствительности к параметрам...

   Варьирование  $\beta$  ( $c=10.0$ ,  $\gamma=0.25$ ):
   Could not create decoration from factory! Running with no decorations.
    $\beta=0.03$ : пик I=24, время=37.87, переболело=116
    $\beta=0.05$ : пик I=174, время=17.87, переболело=751
    $\beta=0.07$ : пик I=264, время=12.25, переболело=924

   Варьирование  $c$  ( $\beta=0.05$ ,  $\gamma=0.25$ ):
    $c=5.0$ : пик I=21, время=7.16, переболело=82
    $c=10.0$ : пик I=174, время=17.87, переболело=751
    $c=15.0$ : пик I=293, время=8.24, переболело=924

   Варьирование  $\gamma$  ( $\beta=0.05$ ,  $c=10.0$ ):
    $\gamma=0.15$ : пик I=376, время=14.6, переболело=941
    $\gamma=0.25$ : пик I=174, время=17.87, переболело=751
    $\gamma=0.35$ : пик I=109, время=20.55, переболело=576
   График чувствительности сохранён: plots/sensitivity_analysis.png

3. Сравнение стохастического и детерминированного выздоровления...
   Сравнительный график сохранён: plots/recovery_comparison.png

4. Оценка производительности...
   Создание модели с N=10000 индивидов...
   0.020031 seconds (176.02 k allocations: 6.978 MiB)

   Бенчмарк sir_run для 10000 индивидов:
   Среднее время: 0.0 секунд
   Минимальное время: 0.0 секунд
   Максимальное время: 0.0 секунд
```

Рисунок 4.1: Запуск кода и его вывод

```

5. Сохранение результатов в CSV...
Результаты сохранены: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/data/sims/sir_990_10_0.05_10_0.25_2026-05-27_18-39-31.csv

=====
ИТОГОВАЯ СТАТИСТИКА
=====
Начальные условия: S=990, I=10, R=0
Параметры:  $\beta=0.05$ ,  $c=10.0$ ,  $\gamma=0.25$ 
 $R_0 = \beta \cdot c / \gamma = 2.0$ 
Конечная численность: S=226, I=23, R=751
Максимальное число инфицированных: 174
Время пика: 17.87
Общая доля переболевших: 75.1%

Работа завершена успешно!

```

Рисунок 4.2: Запуск кода и его вывод

Далее создала литературный код для всех основных файлов. После скомпилировала чистый код, jupyter notebook и quarto с помощью файла с кодом `tangle.jl` (рис. 4.2).

```

zrdagdelen@zrdagdelen-MacBookPro:~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project$ julia --project=. scripts/tangle.jl scripts/sir_des.jl
Генерация из: scripts/sir_des.jl
[ Info: generating plain script file from `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/scripts/sir_des.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/scripts/sir_des/sir_des.jl`
✓ Чистый скрипт: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/scripts/sir_des/sir_des.jl
[ Info: generating markdown page from `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/scripts/sir_des.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/markdown/sir_des/sir_des.qmd`
✓ Quarto: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/markdown/sir_des/sir_des.qmd
[ Info: generating notebook from `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/scripts/sir_des.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/notebooks/sir_des/sir_des.ipynb`
✓ Notebook: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab08/project/notebooks/sir_des/sir_des.ipynb
Готово! Все файлы созданы.

```

Рисунок 4.3: Компиляция `sir_des.jl`

Код из файлов и анализ результатов предоставлен ниже.

5 Обзор sir_des

Данный скрипт является точкой входа для запуска дискретно-событийной SIR-модели. Он выполняет:

1. Базовый запуск модели с параметрами по умолчанию
2. Анализ чувствительности к параметрам β , γ , δ
3. Сравнение стохастической и детерминированной версий выздоровления
4. Оценку производительности для популяции из 10000 индивидов
5. Сохранение результатов в CSV файл
6. Вывод итоговой статистики

5.1 Подключение библиотек и модулей

5.1.1 Активация окружения проекта DrWatson

DrWatson обеспечивает воспроизводимость и управление путями к файлам.

```
#!/usr/bin/env julia

using DrWatson
@quickactivate "project"
```

5.1.2 Подключение ядра модели

Импортируем функции из `sir_model.jl`: - `MakeSIRModel` - создание модели - `MakeSIRModel_deterministic_recovery` - модель с детерминированным выздоровлением - `activate/activate_deterministic` - активация процессов - `sir_run` - запуск симуляции - `out` - сбор результатов

```
include(sourcedir("sir_model.jl"))
```

5.1.3 Импорт дополнительных библиотек

```
using Random          # [random] [random] [seed] [seed] seed
using StatsPlots      # [plots] [plots]
using BenchmarkTools  # [benchmark] [benchmark]
using CSV              # [csv] [csv] [csv] CSV
using Dates            # [dates] [dates] [dates] [dates]
```

5.2 Параметры модели по умолчанию

Определим основные параметры моделирования:

Длительность симуляции - 40 единиц времени

```
tmax = 40.0
```

Начальные условия - 990 восприимчивых, 10 инфицированных, 0 переболевших

```
u0 = [990, 10, 0] # S, I, R
```

Параметры модели - вероятность заражения $\beta=0.05$, частота контактов $c=10.0$, интенсивность выздоровления $\gamma=0.25$

```
p_default = [0.05, 10.0, 0.25] #  $\beta$ ,  $c$ ,  $\gamma$ 
```

Фиксация seed для воспроизводимости

```
Random.seed!(1234)
```

Вывод заголовка программы

```
println("="^60)
println("XXXXXXXXXX-XXXXXXXXXXXX SIR XXXXXXXX")
println("="^60)
```

5.3 Базовый запуск модели

Сначала выполним базовый запуск с параметрами по умолчанию. Это позволит получить эталонную динамику для сравнения.

```
println("\n1. XXXXXXXX XXXXXXXX XXXXXXXX...")
```

Создание модели

```
des_model = MakeSIRModel(u0, p_default)
```

Активация процессов агентов

```
activate(des_model)
```

Запуск симуляции до времени tmax

```
sir_run(des_model, tmax)
```

Сбор результатов в DataFrame

```
data_des = out(des_model)
```

5.3.1 Визуализация базового запуска

Построим график изменения численности S, I, R во времени.

```
plot(data_des.t, [data_des.S data_des.I data_des.R],  
      labels=["S" "I" "R"],  
      xlabel="time",  
      ylabel="population",  
      title="SIR model ( $\beta=0.05$ ,  $c=10.0$ ,  $\gamma=0.25$ ),  
      linewidth=2)
```

Сохранение графика в директорию plots/

```
savefig(plotsdir("sir_des.png"))  
  
println("    sir_des.png: plots/sir_des.png")
```

5.4 Анализ чувствительности к параметрам (Задание 8.5.1)

Проведём анализ влияния каждого параметра на динамику эпидемии. Для каждого набора параметров записываем: - пик инфицированных - время достижения пика - итоговую долю переболевших

```
println("\n2.    sir_des.png: plots/sir_des.png...")
```

5.4.1 Варьирование β (вероятность заражения)

β - вероятность передачи инфекции при контакте. Значения варьируются от 0.03 до 0.07.

```
betas = [0.03, 0.05, 0.07]
results_beta = []

println("\n      ████████████████████ (c=10.0, ρ=0.25):")
for ρ in betas
```

Фиксируем seed для воспроизводимости при каждом запуске

```
Random.seed!(1234)
p = [0, 10.0, 0.25]
m = MakeSIRModel(u0, p)
activate(m)
sir_run(m, tmax)
data = out(m)
```

Вычисление ключевых метрик

```
peak_I = maximum(data.I) # [ ] [ ]
peak_time = data.t[argmax(data.I)] # [ ] [ ]
final_R = data.R[end] # [ ] [ ] [ ]
```



```
push!(results_beta, (x=x, peak_I=peak_I, peak_time=peak_time, final_R=final_R))
println(" x=$x: [ ] I=$(round(peak_I)), [ ]=$(round(peak_time, digits=2))")
end
```


5.4.2 Варьирование c (частота контактов)

c - среднее число контактов индивида в единицу времени. Значения варьируются от 5.0 до 15.0.

```
cs = [5.0, 10.0, 15.0]
results_c = []

println("\n      c (0.05, 0.25):")
for c in cs
    Random.seed!(1234)
    p = [0.05, c, 0.25]
    m = MakeSIRModel(u0, p)
    activate(m)
    sir_run(m, tmax)
    data = out(m)

    peak_I = maximum(data.I)
    peak_time = data.t[argmax(data.I)]
    final_R = data.R[end]

    push!(results_c, (c=c, peak_I=peak_I, peak_time=peak_time, final_R=final_R))
    println("    c=$c: I=$(round(peak_I)), peak_time=$(round(peak_time, digits=2))")
end
```

5.4.3 Варьирование γ (интенсивность выздоровления)

γ - скорость выздоровления (обратная среднему времени болезни). Значения варьируются от 0.15 до 0.35.

```

gammas = [0.15, 0.25, 0.35]
results_gamma = []

println("\n      XXXXXXXXXXXXXXXX  (u=0.05, c=10.0):")
for  in gammas
    Random.seed!(1234)
    p = [0.05, 10.0, ]
    m = MakeSIRModel(u, p)
    activate(m)
    sir_run(m, tmax)
    data = out(m)

    peak_I = maximum(data.I)
    peak_time = data.t[argmax(data.I)]
    final_R = data.R[end]

    push!(results_gamma, (==, peak_I=peak_I, peak_time=peak_time, final_R=final_R))
    println("  ==$: XXXX I=$(round(peak_I)), XXXXX=$(round(peak_time, digits=2))")
end

```

5.4.4 Визуализация результатов чувствительности

Построим графики зависимости пика инфицированных от каждого параметра.

График для β

```

p1 = plot(title="XXXXXXXXXXXXXXXXXXXX  ",
           xlabel="",
           ylabel="XXXX I")
p1 = scatter!([r. for r in results_beta],

```

```
[r.peak_I for r in results_beta],
label="I",
markersize=8)
```

График для c

```
p2 = plot(title="I vs c",
          xlabel="c",
          ylabel="I")
p2 = scatter!([r.c for r in results_c],
              [r.peak_I for r in results_c],
              label="I",
              markersize=8)
```

График для γ

```
p3 = plot(title="I vs \gamma",
          xlabel="\gamma",
          ylabel="I")
p3 = scatter!([r.\gamma for r in results_gamma],
              [r.peak_I for r in results_gamma],
              label="I",
              markersize=8)
```

Объединение графиков в один ряд

```
plot(p1, p2, p3, layout=(1,3), size=(900, 400))
```

Сохранение

```
savefig(plotsdir("sensitivity_analysis.png"))
println("  XXXXX XXXXXXXXXXXXXXXX XXXXXX: plots/sensitivity_analysis.png")
```

5.5 Сравнение стохастической и детерминированной версий (Задание 8.5.2)

Сравним два варианта модели: - **Стохастическая версия**: время выздоровления $\sim \text{Exponential}(1/\gamma)$ - **Детерминированная версия**: время выздоровления = $1/\gamma$ (фиксированное)

```
println("\n3. XXXXXX XXXXXXXXXXXXXXXX X XXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXX...")
```

5.5.1 Стохастическая версия

```
Random.seed!(1234)
m_stoch = MakeSIRModel(u0, p_default)
activate(m_stoch)
sir_run(m_stoch, tmax)
data_stoch = out(m_stoch)
```

5.5.2 Детерминированная версия

```
Random.seed!(1234)
m_det = MakeSIRModel_deterministic_recovery(u0, p_default)
activate_deterministic(m_det)
sir_run(m_det, tmax)
data_det = out(m_det)
```

5.5.3 Сравнительный график

Наложим кривые инфицированных для обеих версий.

```
plot(data_stoch.t, data_stoch.I,
      label="SIR model: Infected",
      xlabel="Time",
      ylabel="Number of individuals",
      linewidth=2,
      linestyle=:solid)

plot!(data_det.t, data_det.I,
      label="SEIR model: Infected",
      linewidth=2,
      linestyle=:dash)

title!("Comparison of SIR and SEIR models")

savefig(plotsdir("recovery_comparison.png"))
println("  SEIR model: plots/recovery_comparison.png")
```

5.6 Оценка производительности (Задание 8.5.3)

Измерим время выполнения модели для популяции из 10000 индивидов. Используем макрос `@benchmark` из пакета `BenchmarkTools`.

```
println("\n4.  SEIR model...")
```

5.6.1 Создание модели большого размера

Популяция: 9990 восприимчивых + 10 инфицированных = 10000 индивидов
Время симуляции сокращено до 10 единиц для ускорения

```
u_large = [9990, 10, 0]
tmax_short = 10.0

println("    XXXXXXXX XXXXXX X N=10000 XXXXXXXX...")
@time begin
    m_large = MakeSIRModel(u_large, p_default)
    activate(m_large)
end
```

5.6.2 Бенчмаркинг

Выполняем 3 прогона для статистической оценки

```
println("\n    XXXXXXXX sir_run XXX 10000 XXXXXXXX:")
bench_result = @benchmark sir_run($m_large, $tmax_short) samples=3
```

Вывод результатов в секундах

```
println("    XXXXXXXX XXXXXX: $(round(median(bench_result).time/1e9, digits=3))s")
println("    XXXXXXXXXXXX XXXXXX: $(round(minimum(bench_result).time/1e9, digits=3))s")
println("    XXXXXXXXXXXXXXXX XXXXXX: $(round(maximum(bench_result).time/1e9, digits=3))s")
```

5.7 Сохранение результатов в CSV (Задание 8.5.4)

Автоматическое сохранение итоговой таблицы с уникальным именем, содержащим параметры запуска.

```
println("\n5.  XXXXXXXXXXXX XXXXXXXXXXXX X CSV...")
```

Генерация временной метки

```
timestamp = Dates.format(now(), "yyyy-mm-dd_HH-MM-SS")
```

Формирование имени файла с параметрами

```
filename = "sir_$(u[1])_$(u[2])_$(p_default[1])_$(p_default[2])_$(p_default[3])"
```

Путь для сохранения

```
csv_path = datadir("sims", filename)
```

Создание директории (если не существует)

```
mkpath(datadir("sims"))
```

Сохранение DataFrame в CSV

```
CSV.write(csv_path, data_des)
println("  XXXXXXXXXXXX XXXXXXXXXXXX: $csv_path")
```

5.8 Итоговая статистика

Вывод сводных результатов моделирования.

```
println("\n" * "="^60)
println("XXXXXXXXXX XXXXXXXXXXXX")
println("="^60)
```

Основные параметры

```
println("Исходные значения: S_0=$(u[1]), I_0=$(u[2]), R_0=$(u[3])")
println("Параметры: beta=$(p_default[1]), c=$(p_default[2]), gamma=$(p_default[3])")
```

Репродуктивное число

```
R0 = p_default[1] * p_default[2] / p_default[3]
println("R_0 = beta * c / gamma = $(round(R0, digits=3))")
```

Конечные значения

```
println("Конечные значения: S=$(data_des.S[end]), I=$(data_des.I[end])")
```

Пик эпидемии

```
println("Максимальное значение I: $(maximum(data_des.I))")
println("Время до пика: $(round(data_des.t[argmax(data_des.I)], digits=2))")
```

Итоговая доля переболевших

```
final_fraction = data_des.R[end] / sum(u) * 100
println("Итоговая доля переболевших: $(round(final_fraction, digits=1))%")

println("\nИтоги моделирования!")
```

5.9 Выводы

На основе выполненного анализа можно сделать следующие выводы:

1. Чувствительность к параметрам:

- Увеличение β и c приводит к росту пика инфицированных и уменьшению времени пика

- Увеличение γ снижает пик и замедляет распространение эпидемии

2. Сравнение версий:

- Детерминированная версия дает более гладкую кривую без флуктуаций
- Стохастическая версия более реалистична для малых популяций

3. Производительность:

- Для популяции 10000 индивидов модель выполняет симуляцию за приблизительно 0.02 секунды (время создания модели)

4. Репродуктивное число:

- При $R_0 > 1$ эпидемия распространяется
- При $R_0 < 1$ эпидемия затухает

6 Вывод

В результате работы реализована дискретно-событийная SIR-модель на Julia. Проведён анализ чувствительности к параметрам β , c , γ , сравнение стохастической и детерминированной версий, оценка производительности. Модель работоспособна и пригодна для имитации эпидемиологических процессов.

Список литературы

1. Discrete-Event System Simulation / J. Banks [и др.]. — 5th. — Pearson, 2010.
2. Julia: A fresh approach to numerical computing / J. Bezanson [и др.] // SIAM Review. — 2017. — Т. 59, № 1. — С. 65—98.
3. *Kermack W. O., McKendrick A. G.* A contribution to the mathematical theory of epidemics // Proceedings of the Royal Society of London. Series A. — 1927. — Т. 115, № 772. — С. 700—721.
4. Лабораторная работа №8: Дискретно-событийное моделирование распространения инфекции (SIR). — 2026. — URL: https://esystem.rudn.ru/pluginfile.php/3094247/mod_resource/content/3/simulation-modeling-lab.pdf#chapter.8.